Object-Oriented Programming in Java

Object-Oriented Programming in Java

1. The Problem: Representing Real Entities with Plain Variables
2. The Core Idea: Mimic the Real World
3. Class vs. Object: Abstract Idea vs. Concrete Instance
4. Creating an Object: the `new` Keyword
5. Memory Model: Stack vs. Heap, and Reference Variables
6. The Dot Operator: Accessing an Object's Data
7. Objects Have Behavior Too: Methods
8. OOP Isn't Limited to Physical Things
9. Naming Conventions
10. Java Is (Almost) Purely Object-Oriented

🌍 Real-Life Examples of OOP in Practice

⭐ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

Object-Oriented Programming in Java

Classes, Objects, the `new` Keyword, and Memory Model

1. The Problem: Representing Real Entities with Plain Variables

Suppose you need to store a student's details: name, age, roll number, college.

```
String name = "Aditya";
int age = 28;
int rollNo = 101;
String college = "IIT Guwahati";
```

This works for **one** student. Now represent a second student, Rohit:

```
String name2 = "Rohit";
int age2 = 28;
int rollNo2 = 102;
String college2 = "IIT Guwahati";
```

Four Concrete Problems This Creates

Problem	Why It Hurts
Too many independent variables	Representing *one* student already needs 4 separate variables — with more students, this multiplies endlessly
No grouping in the code	Nothing in the code says `name`, `age`, `rollNo`, `college` belong *together* — you just have to remember it
Passing to functions is clumsy	A `print(student)` function would need **all 4** variables passed in separately, every single time
No ownership over data	Since the four variables are fully independent, nothing "owns" them as a unit — anyone can change any one of them with no relationship enforced

💡 Easy Hinglish Explanation

> "Problem ye hai ki name, age, rollNo, college — ye chaaro variables aapas mein completely independent hain. Code dekh ke pata hi nahi chalta ki ye chaaro ek hi student ko represent kar rahe hain. Hume koi tareeka chahiye jisse in sabko ek saath, ek unit ki tarah rakh sakein."

Object-Oriented Programming (OOP) exists specifically to solve these four problems.

---

2. The Core Idea: Mimic the Real World

> **Definition:** OOP is a programming **paradigm** (a way of thinking about and structuring code) that says: to solve a real-world problem well, first *mimic* the real world inside your program.

In the real world, a student isn't four scattered facts — it's **one entity** that happens to *have* a name, age, roll number, and college. OOP calls any such real-world entity an **object**.

> **Definition:** An **object** is anything in the real world — living or non-living — that has some **characteristics** (data) and, often, some **behaviors** (actions it can perform).

So instead of four independent variables, OOP says: put all four inside **one single block**, exactly like the real world groups them inside one student.

📌 **Important:** This "mimicking" doesn't mean copying *every* real-world detail. If you're building a college application, you only model the characteristics relevant to your problem (name, age, roll number, college) — not everything a real student has (hobbies, height, favorite food). You focus only on what your program actually needs.

---

3. Class vs. Object: Abstract Idea vs. Concrete Instance

Before writing any code, there's an important distinction: when you say "student," you might mean two different things.

Term	Meaning	Example
Class	An abstract *idea* of what a student looks like — a **blueprint**	"A student has a name, age, roll number, and college" (no specific person)
Object	One concrete, actual instance of that idea	"Arjun" is a student; "Rohit" is a student — each with their own actual values

> **Definition:** A **class** is the blueprint that defines what characteristics and behaviors an object *will* have — but a class itself holds **no actual data**. An **object** is a real instance, created from that blueprint, that holds **actual values** in memory.

Defining a Class in Java

```
class Student {
    String name;
    int age;
    int rollNum;
    String college;
}
```

This says: *"Any Student object will have these four characteristics."* No memory for any actual student has been created yet — this is purely the blueprint.

⭐ **Exam Tip:** A very common interview question is *"what's the difference between a class and an object?"* — the cleanest answer: **a class is a blueprint; an object is an instance built from that blueprint, holding real data.**

---

4. Creating an Object: the `new` Keyword

Creating an object is a **two-step process**, exactly like declaring and defining any variable:

```
Student s1;              // Step 1: Declaration
s1 = new Student();       // Step 2: Definition (memory actually allocated)

// Or combined in one line, as usual:
Student s1 = new Student();
```

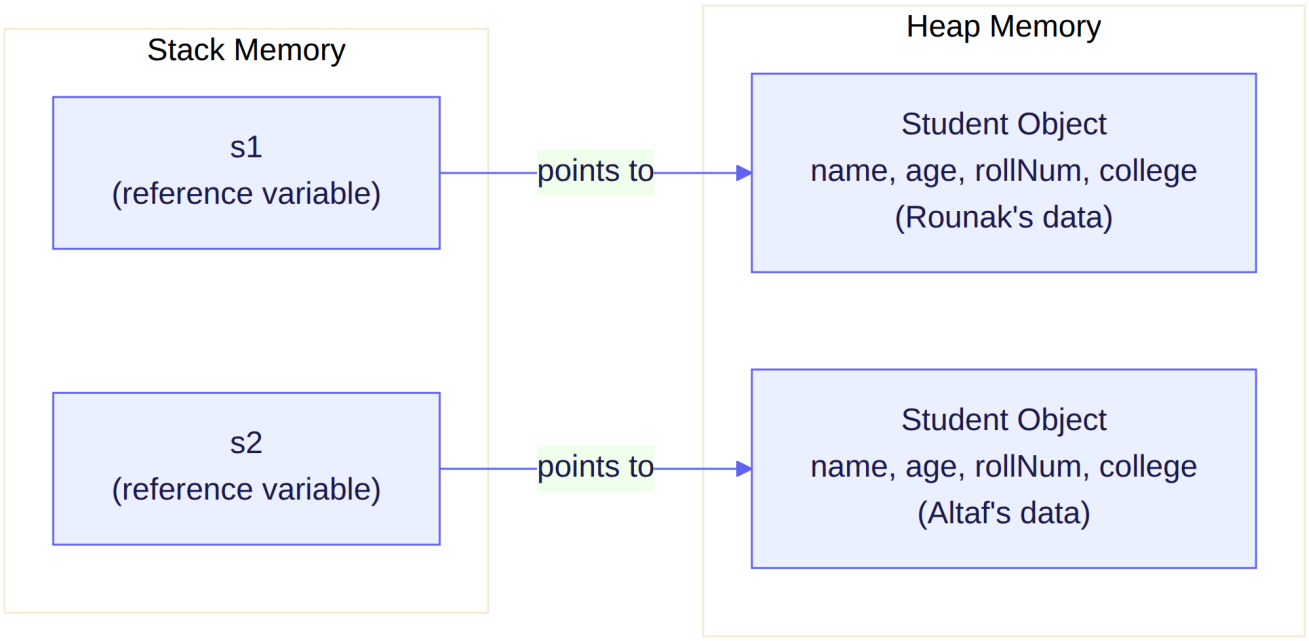
Part	Meaning
`Student` (before `s1`)	The **type** of the reference variable — this variable will point to a `Student` object
`s1`	The reference variable's name
`new`	Allocates actual memory (in the **heap**) for a new `Student` object
`Student()`	Calls the class's constructor to build the object *(constructors are covered in a later, dedicated lecture)*

📌 **Important:** `Student` here is a **non-primitive, user-defined data type** — created by *you*, unlike built-in primitives (`int`, `boolean`, etc.). This is exactly what "non-primitive" meant back when arrays were introduced: any type that isn't one of Java's 8 built-in primitives is non-primitive, and objects are the general case of that idea.

---

5. Memory Model: Stack vs. Heap, and Reference Variables

This directly extends the stack/heap model from arrays: **primitives store values directly on the stack; non-primitives store a reference on the stack, pointing to the actual data in the heap.**



Step-by-step, for `Student s1 = new Student();`:

1. `new Student()` allocates a block of memory in the **heap**, sized to hold all of `Student`'s fields (`name`, `age`, `rollNum`, `college`)
2. This heap block's **starting address** (e.g., `1001`) is what gets returned
3. That address is stored in `s1` — and `s1` itself lives in the **stack**

⚠ **Common Mistake — a very common interview trap:** Beginners often think `s1` *is* the object. It isn't. **`s1` is a reference variable** — it only stores the *address* of the object, not the object itself. The actual object (its data) lives separately, in the heap, and has **no name of its own** — it's only ever reached *through* a reference variable like `s1`.

💡 **Easy Hinglish Explanation**

> "`s1` khud object nahi hai — `s1` sirf ek reference variable hai jo heap memory mein bane object ka address store karta hai. Object khud heap mein hota hai, aur uska koi naam nahi hota — hum usko sirf reference variable ke through hi access kar sakte hain."

Creating a second object works the same way, allocating a **separate** block in the heap:

```
        Student s2 = new Student();
```

Now `s1` and `s2` are two independent reference variables in the stack, each pointing to their own separate `Student` object in the heap.

---

6. The Dot Operator: Accessing an Object's Data

Once an object exists, its fields are accessed using the **dot (`.`) operator**:

```
s1.name = "Rounak";
s1.age = 22;
s1.rollNum = 161;
s1.college = "Noida International University";

s2.name = "Altaf";
s2.age = 23;
s2.rollNum = 162;
s2.college = "Galgotia University";
```

> **Definition:** The dot operator "walks" from a reference variable to the object it points to, then lets you access or assign any field inside that object.

Reading a value works the same way:

```
System.out.println(s1.name);   // Rounak
```

📌 **Important:** Since `s1` and `s2` point to **separate** heap objects, changing `s1.name` has **zero effect** on `s2.name` — each object's fields are completely independent, exactly like two different real students having their own separate names.

---

7. Objects Have Behavior Too: Methods

An object isn't just a data container — it can also perform **actions**, defined as **methods** inside the class:

```
class Student {
    String name;
    int age;
    int rollNum;
    String college;

    void markAttendance() {
        System.out.println("Attendance marked by " + name);
    }

    void print() {
        System.out.println(name + "," + age + "," + rollNum + "," + college);
    }
}
```

Calling these methods, using the same dot operator:

```
s1.markAttendance();   // Attendance marked by Rounak
s2.markAttendance();   // Attendance marked by Altaf
s1.print();
s2.print();
```

⭐ **Exam Tip:** Notice `markAttendance()` uses `name` directly, with **no parameter needed** for it. Because this method belongs to the object itself, it automatically has access to *that specific object's* own `name` — it doesn't need to be told whose name to print.

Full runnable example (from the class demo):

```
        public class OopsIntro {
            public static void main(String[] args) {
                Student s1 = new Student();
                Student s2 = new Student();

                s1.name = "Rounak";  s1.age = 22;  s1.rollNum = 161;  s1.college = "Noida
International University";
                s2.name = "Altaf";   s2.age = 23;  s2.rollNum = 162;  s2.college =
"Galgotia University";

                s1.markAttendance();
                s2.markAttendance();
                s1.print();
                s2.print();
            }
        }

        class Student {
            String name;
            int age;
            int rollNum;
            String college;

            void markAttendance() {
                System.out.println("Attendance marked by " + name);
            }
            void print() {
                System.out.println(name + "," + age + "," + rollNum + "," + college);
            }
        }
```

Output:

```
Attendance marked by Rounak
Attendance marked by Altaf
Rounak,22,161,Noida International University
Altaf,23,162,Galgotia University
```

8. OOP Isn't Limited to Physical Things

Objects don't need to be physical, living entities. Anything with characteristics and behaviors qualifies:

Real-World Entity	Characteristics	Behaviors
`Animal`	`type`, `numberOfLegs`	`makeSound()`, `eat()`
`BankAccount` (abstract, non-physical)	balance, account number	`deposit()`, `withdraw()`
`Location` (abstract, non-physical)	latitude, longitude	—

📌 **Important:** The same real-world entity (like a **Human**) can be modeled as *different* classes depending on the problem you're solving — a `Student` class for a college app, an `Employee` class for a payroll app, a `SonDaughter` class for a family-tree app. You only model the slice of reality that's relevant to your specific problem.

9. Naming Conventions

These are **conventions** (widely followed good practice), not compiler-enforced rules — code still compiles if you ignore them, but following them is standard professional practice:

Element	Convention	Example
Variables (primitive or reference)	**camelCase** — first word lowercase, each next word capitalized	`firstName`, `rollNum`
Class names	**PascalCase** — starts with a capital letter	`Student`, `Human`, `BankAccount`
Method names	**camelCase**	`markAttendance()`, `print()`

💡 Easy Hinglish Explanation

> "CamelCase mein pehla word chhote letter se shuru hota hai, aur uske baad har naye word ka pehla letter capital hota hai — jaise `firstName`. PascalCase mein har word capital letter se shuru hota hai, jaise class ka naam `Student`."

---

10. Java Is (Almost) Purely Object-Oriented

Every single Java program you've written so far has actually been using classes — even the "boilerplate" you were told to ignore early on:

```
public class Demo {
    public static void main(String[] args) {
        // your code
    }
}
```

`Demo` is itself a class — no different in principle from the `Student` class built in this lecture. The `main` method is just a method living inside that class, exactly like `markAttendance()` lives inside `Student`.

📌 **Important:** In Java, you **cannot** write any code without first creating a class — there's no way to write "loose" code outside of one. This is why Java is described as **"almost" fully object-oriented** — nearly everything must live inside a class.

🌍 Real-Life Examples of OOP in Practice

- **E-commerce apps** model `Product`, `Cart`, and `Order` as objects — each `Product` has characteristics (price, name, stock) and behaviors (`applyDiscount()`), exactly like `Student` here.
- **Banking apps** model `Account` objects — balance and account number as characteristics, `deposit()` / `withdraw()` as behaviors — matching the abstract "BankAccount" example above almost exactly.
- **Games** model each character or enemy as an object — health, position, and inventory as characteristics; `attack()`, `move()`, `takeDamage()` as behaviors.
- **Ride-sharing apps** (Uber/Ola) model a `Driver` and a `Ride` as separate objects, each with their own independent data — mirroring how `s1` and `s2` here hold completely separate student data despite coming from the same `Student` blueprint.

⭐ One-Page Revision

Core Definitions

- **Object** = a real-world entity with characteristics (data) and behaviors (methods)
- **Class** = the abstract blueprint defining what an object's characteristics/behaviors will be; holds no actual data itself
- **Reference variable** = a stack-stored variable holding the heap address of an actual object (not the object itself)
- **Dot operator (`.`)** = accesses a specific field or method inside the object a reference variable points to

Key Syntax

```
class Student {                 // blueprint
    String name; int age;
    void print() { ... }
}
Student s1 = new Student();     // create an object (2-step process)
s1.name = "Rounak";             // access/assign a field
s1.print();                     // call a method
```

Key Rules

- `new` always allocates in the **heap**; the reference variable itself lives in the **stack**
- Each object created via `new` is completely independent — changing one never affects another
- camelCase for variables/methods; PascalCase for class names
- Every line of Java code must live inside a class — there's no "loose" code
- Objects can model abstract, non-physical concepts too (BankAccount, Location) — not just physical entities

📝 Practice Questions

Basic

1. What is the difference between a class and an object?
2. What does the `new` keyword actually do in memory?
3. Is `s1` in `Student s1 = new Student();` the object itself? Explain.

Medium 4. Write a `Car` class with fields `brand`, `model`, `year`, and a method `startEngine()` that prints a message including the brand. 5. Create two `Car` objects with different values and call `startEngine()` on each. Explain why their outputs differ even though both come from the same class. 6. Why does `markAttendance()` not need `name` passed in as a parameter?

Interview 7. Explain, step by step, what happens in memory (stack and heap) when `Student s1 = new Student();` executes. 8. Why is Java described as "almost" fully object-oriented rather than "completely" object-oriented? 9. Give two examples of objects that represent abstract, non-physical concepts, and explain what characteristics/behaviors they might have. 10. If `Student s2 = s1;` (no `new`) were written, would `s2` be a new object or point to the same one as `s1`? *(Think about what a reference variable actually stores.)*

🔥 Quick FAQ

Does every class need a `main` method? No — only the class you actually run needs one. Other classes (like `Student` here) just define blueprints and get *used* by a class that has `main`.

Can an object change another object's data directly? Only through that object's own reference variable and dot operator (e.g., `s2.name = "..."`) — one object never automatically affects another's fields.

Is `String` also a class? Yes — `String` is itself a non-primitive, object-based type in Java, which is why it's declared and used differently from primitives like `int` or `boolean`.